

# Zero to Hero Study Handbook: pytorch-distilbert-complaint-classifier

## Module 1: Foundations & Architecture

### 1.1 What This Project Does

This project fine-tunes DistilBERT to classify short banking support queries into 77 intent classes (for example, 'card\_arrival', 'declined\_transaction', 'extra\_charge\_on\_statement'), using the Banking77 dataset.

Primary use cases in this repository:

- Train and evaluate a transformer-based intent classifier end-to-end in a notebook workflow.
- Compare transformer performance against a classical baseline ('TF-IDF + LogisticRegression').
- Run production-style CLI inference for single text or batch files via 'inference.py'.

The source of truth for the notebook content is 'generate\_notebook.py', which generates 'pytorch\_distilbert\_complaint\_classifier.ipynb'.

### 1.2 Core Paradigms and Patterns Used Here

Definitions first, then where they appear:

1. **\*\*Supervised multi-class classification\*\***

The model maps one input text to exactly one label among 77 classes.

Where: notebook code generated in 'generate\_notebook.py' ('NUM\_LABELS', 'id2label', 'label2id').

2. **\*\*Transfer learning / fine-tuning\*\***

A pretrained language model ('distilbert-base-uncased') is adapted to this specific intent task with a new classification head.

Where: 'build\_model()' in generated notebook code.

3. **\*\*Functional pipeline style\*\***

Processing is split into small functions that pass explicit data structures ('set\_seed', 'encode', 'make\_optimizer', 'evaluate', 'metrics\_from\_logits', notebook 'predict'; and in CLI 'load\_model', 'read\_inputs', 'predict').

4. **\*\*Configuration via dataclass\*\***

Training hyperparameters and paths are centralized in 'Config' dataclass (for example 'max\_len', 'epochs', 'lr', 'warmup\_ratio', 'ckpt\_dir', 'fig\_dir').

Where: generated notebook code in 'generate\_notebook.py'.

5. **\*\*Artifact-contract pattern\*\***

Training produces artifacts consumed by inference:

- 'checkpoints/best\_model.pt' (raw PyTorch checkpoint dict)
- 'checkpoints/best\_model/' (HuggingFace 'save\_pretrained' directory for model+tokenizer)

Inference loads 'checkpoints/best\_model/' by default.

6. **\*\*Selective prediction pattern\*\***

The notebook evaluates confidence-based routing (coverage vs. accuracy) to support human fallback on low-confidence predictions.

Where: confidence analysis section in generated notebook code.

### 1.3 Architecture Description (Components + Interactions)

Main components:

- 'generate\_notebook.py'

Builds notebook cells and writes 'pytorch\_distilbert\_complaint\_classifier.ipynb'.

- 'pytorch\_distilbert\_complaint\_classifier.ipynb'

Runs the full ML lifecycle:

data load -> split -> tokenization -> dataloaders -> model training -> checkpointing -> test evaluation -> error/confidence analysis -> baseline comparison.

- 'inference.py'

Standalone prediction CLI. Loads saved HF model+tokenizer and returns top-k intent probabilities for text input(s).

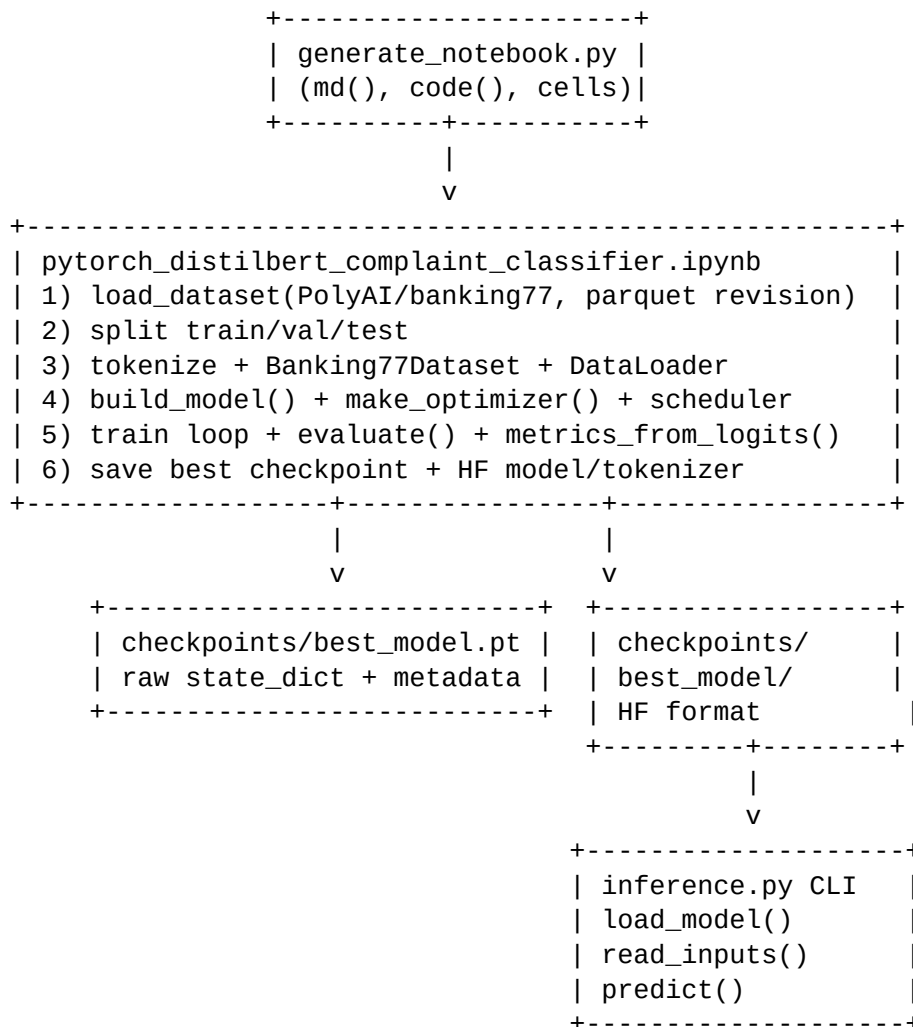
- 'pyproject.toml'

Declares Python/package dependencies and pins 'torch' to the 'pytorch-cu128' UV index.

- Runtime artifact folders (created during notebook execution): 'checkpoints/', 'figures/'.

ASCII flow diagram:

```
'''text
```



'''

## ## Module 2: Repository Map

Focus files new contributors should understand first:

| File/Directory Path                             | Primary Responsibility                                                         | Key Classes/Functions                                                                                                                                                                                                     | Important Configs/Variables                                                                                                                                                                                 |
|-------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'README.md'                                     | Project-level explanation, setup, run commands, expected artifacts and metrics | N/A (documentation)                                                                                                                                                                                                       | Dataset split counts, training settings table, run commands, artifact paths                                                                                                                                 |
| 'pyproject.toml'                                | Python project metadata and dependency specification                           | N/A                                                                                                                                                                                                                       | '[project].requires-python = ">=3.12.10"', dependency list, '[tool.uv.sources]', '[tool.uv.index]' with 'pytorch-cu128'                                                                                     |
| 'python-version'                                | Local Python version pin                                                       | N/A                                                                                                                                                                                                                       | '3.12.10'                                                                                                                                                                                                   |
| 'gitignore'                                     | Excludes generated/runtime artifacts from git                                  | N/A                                                                                                                                                                                                                       | Ignores '.venv/', 'checkpoints/best_model/', 'checkpoints/*.pt', 'figures/*.png', 'data/', etc.                                                                                                             |
| 'generate_notebook.py'                          | Source-of-truth notebook generator using 'nbformat'                            | 'md', 'code' (generator helpers); embedded notebook functions include 'set_seed', 'encode', 'Banking77Dataset', 'build_model', 'make_optimizer', 'evaluate', 'metrics_from_logits', 'show_confusions', notebook 'predict' | 'SEED', 'Config' fields ('model_name', 'dataset', 'dataset_rev', 'max_len', 'batch_size', 'eval_batch', 'epochs', 'lr', 'weight_decay', 'warmup_ratio', 'max_grad_norm', 'val_size', 'ckpt_dir', 'fig_dir') |
| 'pytorch_distilbert_complaint_classifier.ipynb' | Executed training/evaluation artifact with outputs, plots, and metrics         | Same runtime functions generated by 'generate_notebook.py'; sequential notebook execution cells                                                                                                                           | Recorded outputs (for example: best val macro-F1 '0.9208', test accuracy '0.9234')                                                                                                                          |
| 'inference.py'                                  | CLI inference entrypoint for single and batch predictions                      | 'load_model', 'predict', 'read_inputs', 'main'                                                                                                                                                                            | CLI args: '--text', '--input', '--model-dir', '--output', '--top-k', '--max-len', '--device'; defaults include 'model-dir=checkpoints/best_model', 'max_len=64', 'top_k=3'                                  |
| 'uv.lock'                                       | Locked dependency resolution for reproducibility                               | N/A                                                                                                                                                                                                                       | Exact pinned dependency graph for 'uv'                                                                                                                                                                      |

## ## Module 3: Core Execution Flows

### ### Flow A: Notebook Generation ('generate\_notebook.py')

Entry point behavior:

1. Defines helper wrappers:

- 'md(src: str) -> nbf.NotebookNode'
- 'code(src: str) -> nbf.NotebookNode'

2. Builds a 'cells' list containing markdown/code cell payloads for the full tutorial pipeline

3. Creates notebook object and metadata:

- 'nb = nbf.v4.new\_notebook()'
- 'nb["cells"] = cells'
- kernelspec and language info ('python 3.12.10')

4. Writes to:

- 'Path(\_\_file\_\_).parent / "pytorch\_distilbert\_complaint\_classifier.ipynb"'

Short code fragment (actual):

```

'''python
nb = nbf.v4.new_notebook()
nb["cells"] = cells
out_path = Path(__file__).parent / "pytorch_distilbert_complaint_classifier.ipynb"
nbf.write(nb, out_path)
'''

```

Input/Output shape:

- Input: no external runtime inputs (beyond script source itself).
- Output: one notebook file containing all training/evaluation logic and narrative cells.

---

### ### Flow B: Training + Evaluation Pipeline (Notebook Runtime Path)

This flow is implemented as code inside notebook cells generated by 'generate\_notebook.py'.

#### ##### Step 1: Reproducibility and device policy

- 'set\_seed()' sets seeds for Python, NumPy, Torch CPU/CUDA.
- Device selection: CUDA if available else CPU.
- Mixed precision policy:
  - bf16 when 'torch.cuda.is\_bf16\_supported()'
  - else fp16 + 'GradScaler'
  - else fp32 on CPU

#### ##### Step 2: Configuration

- 'Config' dataclass centralizes run settings.
- Notable defaults:
  - 'model\_name="distilbert-base-uncased"'
  - 'dataset="PolyAI/banking77"'
  - 'dataset\_rev="refs/convert/parquet"'
  - 'max\_len=64', 'batch\_size=32', 'eval\_batch=64'
  - 'epochs=8', 'lr=5e-5', 'warmup\_ratio=0.1'
  - 'weight\_decay=0.01', 'max\_grad\_norm=1.0'
  - 'val\_size=0.10', 'seed=42'
  - 'ckpt\_dir=Path("checkpoints")', 'fig\_dir=Path("figures")'

#### ##### Step 3: Data loading and label maps

- Loads dataset:
  - 'raw = load\_dataset(cfg.dataset, revision=cfg.dataset\_rev)'
- Constructs label maps from 'ClassLabel':
  - 'id2label: dict[int, str]'
  - 'label2id: dict[str, int]'
- Data row shape from dataset:
  - '{"text": <str>, "label": <int>}'
- Committed notebook output shows:
  - 'train: 10,003', 'test: 3,080', 'Num intents: 77'

#### ##### Step 4: Split and tokenize

- Creates validation split from train with stratification:
  - 'train\_test\_split(..., test\_size=cfg.val\_size, stratify=trainlbls\_all,

```

    random_state=cfg.seed)'
- Uses 'encode(texts)' wrapper around tokenizer with truncation to 'max_len'.
- Committed notebook output split counts:
  - 'train: 9,002', 'val: 1,001', 'test: 3,080'

#### Step 5: Dataset/DataLoader construction
- 'Banking77Dataset.__getitem__' returns:

'''python
{
    "input_ids": <list[int]>,
    "attention_mask": <list[int]>,
    "labels": <int>
}
'''

- 'DataCollatorWithPadding(..., return_tensors="pt")' produces padded batch tensors.
- Committed notebook output example batch:
  - keys: ['input_ids', 'attention_mask', 'labels']
  - 'input_ids' shape: '(32, 53)' (batch, padded_len)
  - 'labels' shape: '(32,)'

#### Step 6: Model, optimizer, scheduler
- 'build_model()' initializes 'AutoModelForSequenceClassification' with:
  - 'num_labels=NUM_LABELS'
  - 'id2label', 'label2id'
- 'make_optimizer()' builds AdamW parameter groups:
  - with weight decay for most params
  - zero decay for '"bias"' and '"LayerNorm.weight"'
- Scheduler:
  - 'get_linear_schedule_with_warmup(optimizer, warmup_steps, total_steps)'

#### Step 7: Training loop + checkpoint selection
- For each epoch:
  - forward under 'autocast'
  - backward ('GradScaler' branch for fp16)
  - 'clip_grad_norm_'
  - optimizer step + scheduler step
- Per epoch validation:
  - 'evaluate(model, val_loader)' returns '(loss, logits, labels)'
  - 'metrics_from_logits(...)' returns:
    - 'accuracy'
    - 'macro_f1'
    - 'weighted_f1'
- Checkpoint criterion:
  - save only when current 'macro_f1' exceeds 'best_f1'

Raw checkpoint payload keys ('best_model.pt'):
- 'epoch'
- 'model_state_dict'
- 'id2label'
- 'label2id'

```

- 'model\_name'
- 'max\_len'
- 'val\_metrics'

Also saves HF format:

- 'model.save\_pretrained(hf\_dir)'
- 'tokenizer.save\_pretrained(hf\_dir)'

Committed notebook outputs include:

- 'Total optimisation steps: 2256 (warmup 225)'
- 'Best val macro-F1 = 0.9208' (epoch 5)

##### Step 8: Test evaluation + analysis + baseline

- Reloads best checkpoint ('torch.load') into 'best\_model'.
- Runs held-out test metrics.
- Committed outputs:
  - 'loss: 0.2989'
  - 'accuracy: 0.9234'
  - 'macro-F1: 0.9233'
  - 'weighted-F1: 0.9233'
- Computes:
  - per-class report ('classification\_report')
  - confusion matrix ('confusion\_matrix')
  - top confusion pairs
  - confidence/coverage analysis
  - classical baseline ('TfidfVectorizer' + 'LogisticRegression')

Committed baseline comparison output:

- 'TF-IDF + LogReg': accuracy '0.8899', macro-F1 '0.8903', weighted-F1 '0.8903'
- 'DistilBERT (fine-tuned)': accuracy '0.9234', macro-F1 '0.9233', weighted-F1 '0.9233'

---

### Flow C: CLI Inference ('inference.py')

Entrypoint: 'main()'

Step-by-step:

1. Parse CLI args with mutually exclusive source:
  - '--text' (single query) or '--input' (file path).
2. Resolve device:
  - 'auto' => CUDA if available else CPU.
3. 'load\_model(Path(args.model\_dir), device)':
  - exits if model directory missing
  - loads tokenizer/model from HF directory
4. Read inputs:
  - '--text': one-item list
  - '--input': 'read\_inputs(path)'
  - '.csv' requires 'text' column
  - otherwise treated as line-based text file
5. Run 'predict(...)':
  - tokenization with truncation/padding

- forward pass -> logits -> softmax
  - top-k selection (clipped to number of labels)
6. Print per-query top-k predictions.
7. Optional CSV output when '--output' is provided.

'predict(...)' return type shape:

- 'List[List[Tuple[str, float]]]'
- Outer list length = number of input texts
- Inner list length = 'top\_k' (or num\_labels if smaller)
- Tuple format = '(intent\_name, probability)'

Batch CSV output schema from 'main()':

- required columns always written:
  - 'text'
  - 'predicted\_intent'
  - 'confidence'
- additional per-rank columns:
  - 'top{rank}\_intent'
  - 'top{rank}\_prob'

For default '--top-k 3', output includes:

- 'top1\_intent', 'top1\_prob'
- 'top2\_intent', 'top2\_prob'
- 'top3\_intent', 'top3\_prob'

## ## Module 4: Setup & Run Guide

### ### 4.1 Clean-Machine Prerequisites

- Shell context: README commands are provided in bash style.
- Python: '3.12.10' (from '.python-version', and '>=3.12.10' in 'pyproject.toml').
- Package/tool manager: 'uv' (project is configured for 'uv' workflow).
- Optional acceleration: NVIDIA GPU (CUDA 12.8 wheel index configured for torch).

### ### 4.2 Dependency Installation

Typical sequence from README:

```
'''bash
git clone https://github.com/pypi-ahmad/pytorch-distilbert-complaint-classifier.git
cd pytorch-distilbert-complaint-classifier
uv sync
'''
```

What 'uv sync' uses:

- 'pyproject.toml' dependency list:
  - 'torch', 'transformers', 'datasets', 'accelerate', 'numpy', 'pandas', 'scikit-learn', 'matplotlib', 'seaborn', 'jupyter', 'ipykernel', 'nbformat', 'tqdm'
- 'tool.uv' torch source override:
  - 'pytorch-cu128' index ('https://download.pytorch.org/whl/cu128')

### ### 4.3 Configuration and Environment Variables

- '.env' keys: none required by current codebase.
- Environment variables in code: none referenced in 'generate\_notebook.py' or 'inference.py'.

- Primary config surfaces:
  - 'pyproject.toml' for dependency/runtime requirements
  - 'Config' dataclass in notebook code for training hyperparameters/paths
  - CLI flags in 'inference.py' for inference-time behavior

#### ### 4.4 Typical Command Sequences

1. Regenerate notebook from source script:

```
'''bash
uv run python generate_notebook.py
'''
```

2. Run training/evaluation notebook:

```
'''bash
uv run jupyter lab pytorch_distilbert_complaint_classifier.ipynb
'''
```

Or headless execution:

```
'''bash
uv run jupyter nbconvert --to notebook --execute --inplace
pytorch_distilbert_complaint_classifier.ipynb
'''
```

3. Run inference after training artifacts exist:

```
'''bash
uv run python inference.py --text "My card still hasn't arrived"
uv run python inference.py --input queries.txt --top-k 3
uv run python inference.py --input queries.csv --output predictions.csv
'''
```

#### ### 4.5 Migrations / Seeding / External Services

- Database migrations: none (no DB in this repository).
- Seeding scripts: none beyond ML random seed setup in notebook code.
- External downloads happen at runtime when executing notebook/inference:
  - HuggingFace model/tokenizer ('distilbert-base-uncased')
  - Banking77 dataset from 'PolyAI/banking77' parquet revision.

#### ### 4.6 Export This Handbook to PDF

Example command (not executed in this analysis run):

```
'''bash
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
'''
```

### ## Module 5: Study Plan & Practice Exercises

#### ### 5.1 Ordered Study Plan



1. Start with 'README.md'

Goal: understand problem framing, expected metrics, and run workflow.

2. Read 'pyproject.toml' and '.python-version'

Goal: understand runtime/dependency contract and CUDA torch index pinning.

3. Read 'inference.py' top-to-bottom

Goal: understand the serving/prediction contract and exact I/O schema.

4. Read 'generate\_notebook.py' structure

Goal: understand why notebook is generated, and how 'md()' + 'code()' compose a reproducible notebook.

5. Deep read notebook runtime sections in generated code (inside 'generate\_notebook.py')

Order:

- Reproducibility/device
- 'Config'
- data loading/splitting/tokenization
- 'Banking77Dataset' + dataloaders
- model/optimizer/scheduler
- train/evaluate/checkpoint loop
- test evaluation/confusion/confidence/baseline

6. Open 'pytorch\_distilbert\_complaint\_classifier.ipynb' outputs

Goal: connect code paths to concrete numeric outputs and artifacts.

## ### 5.2 Practice Exercises (with Solution Outlines)

### #### Exercise 1

Question: Which function in 'inference.py' enforces that a CSV input must contain a 'text' column, and what happens if it is missing?

Solution outline:

- Function: 'read\_inputs(path: Path)'.
- Branch: when 'path.suffix.lower() == ".csv"'.
- It checks '"text" in df.columns'; if missing, it calls 'sys.exit(...)' with an error message listing found columns.

### #### Exercise 2

Question: Trace how label IDs become readable intent names during inference.

Solution outline:

- Notebook training code builds 'id2label' and passes it into model config via 'build\_model()'.
- Saved HF artifacts preserve this mapping.
- 'inference.py' reads 'model.config.id2label' in 'predict()'.
- Top-k indices are converted to strings with 'id2label[int(i)]'.

### #### Exercise 3

Question: What is the exact checkpoint selection rule in training?

Solution outline:

- In training loop, after validation metrics, condition is:

- 'if m["macro\_f1"] > best\_f1:'
- So best model is selected strictly by validation macro-F1, not accuracy and not loss.

#### #### Exercise 4

Question: What keys are stored in 'checkpoints/best\_model.pt'?

Solution outline:

- 'epoch'
- 'model\_state\_dict'
- 'id2label'
- 'label2id'
- 'model\_name'
- 'max\_len'
- 'val\_metrics'

#### #### Exercise 5

Question: Explain the difference between 'checkpoints/best\_model.pt' and 'checkpoints/best\_model/'.

Solution outline:

- 'best\_model.pt': raw torch checkpoint dict, useful for manual load/inspection.
- 'best\_model/': HuggingFace 'save\_pretrained' directory containing model and tokenizer files.
- 'inference.py' default loader uses HF directory ('--model-dir' default is 'checkpoints/best\_model').

#### #### Exercise 6

Question: In the notebook, what data structure does 'Banking77Dataset.\_\_getitem\_\_' return, and why does it matter for the model call?

Solution outline:

- Returns dict with keys 'input\_ids', 'attention\_mask', 'labels'.
- This shape matches expected keyword arguments for 'AutoModelForSequenceClassification' forward pass ('model(\*\*batch)').

#### #### Exercise 7

Question: Why does this project use 'dataset\_rev="refs/convert/parquet"' when loading Banking77?

Solution outline:

- Explicitly documented in README and notebook markdown:
  - 'datasets >= 5' dropped script-based loader support for the original dataset repo.
  - parquet revision is used for compatibility while keeping label metadata.

#### #### Exercise 8

Question: What are the default inference-time controls for sequence length, number of predictions, and execution device?

Solution outline:

- In 'inference.py' parser defaults:
  - '--max-len': '64'
  - '--top-k': '3'
  - '--device': '"auto"' (chooses CUDA if available else CPU)

### ### 5.3 Learner Self-Verification Checklist

Use this checklist after finishing all modules:

- Can you explain the end-to-end artifact pipeline from 'generate\_notebook.py' to 'inference.py'?
- Can you describe the exact training selection criterion and why macro-F1 is used?
- Can you list all fields in the raw checkpoint dict and explain what consumes each?
- Can you state the exact input requirements for 'inference.py' batch CSV mode?
- Can you reconstruct the shape of a training batch ('input\_ids', 'attention\_mask', 'labels')?
- Can you explain how 'id2label'/'label2id' are created and propagated into inference outputs?
- Can you identify where torch dependency source pinning is configured and why?
- Can you explain the confidence-threshold routing idea implemented in the notebook analysis cells?
- Can you reproduce the major held-out test metrics reported in the committed notebook outputs?
- Can you point to the single place where training hyperparameters are centralized ('Config')?